Question 7: Create a Jenkinsfile defining pipeline stages (build, test, deploy) and demonstrate a declarative or scripted pipeline execution.

A declarative Jenkinsfile was written with build, test and deploy stages and executed in Jenkins. Both runs (manual and SCM-triggered) finished green.

Jenkinsfile (declarative pipeline)

```
// Declarative pipeline for the DevOps Lab 1 flask app
pipeline {
    agent any

    stages {
        stage("Checkout") {
            steps {
                echo "Pulling source code from GitHub..."
                checkout scm
            }
        }
        stage("Build") {
            steps {
                echo "Installing dependencies..."
                sh "python3 -m venv venv"
                sh "./venv/bin/pip install -r app/requirements.txt"
            }
        }
        stage("Test") {
            steps {
                echo "Running unit tests..."
                sh "./venv/bin/python -m pytest tests/ -v"
            }
        }
        stage("Package") {
            steps {
                sh "mkdir -p dist && tar czf dist/app-build-${BUILD_NUMBER}.tar.gz app tests"
                archiveArtifacts artifacts: "dist/*.tar.gz", fingerprint: true
            }
        }
        stage("Deploy") {
            steps {
                echo "Deploying app to /opt/devops-lab1 and restarting service..."
                sh "sudo bash /var/lib/jenkins/workspace/devops-lab-pipeline/scripts/deploy.sh"
            }
        }
    }

    post {
        success {
            echo "Pipeline finished: BUILD SUCCESS"
        }
        failure {
            echo "Pipeline finished: BUILD FAILED"
        }
    }
}
```

Stage description

Stage	What it does	Key step
Checkout	Pulls the repo from GitHub	checkout scm
Build	Creates a virtualenv and installs Flask	pip install -r app/requirements.txt
Test	Runs the pytest unit tests	python -m pytest tests/ -v

Package	Tars the source into dist/app-build-N.tar.gz	archiveArtifacts
Deploy	Copies app to /opt/devops-lab1, restarts systemd service	scripts/deploy.sh

Execution evidence

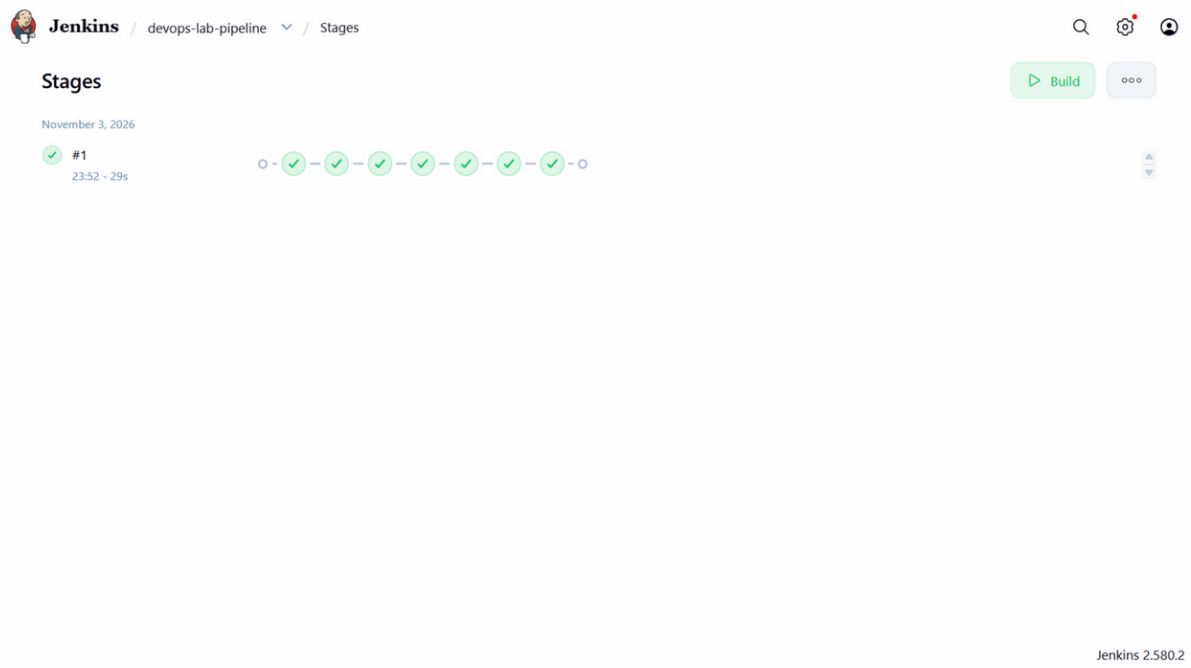


Fig: Stage view of build #1 - 7 stages, all green

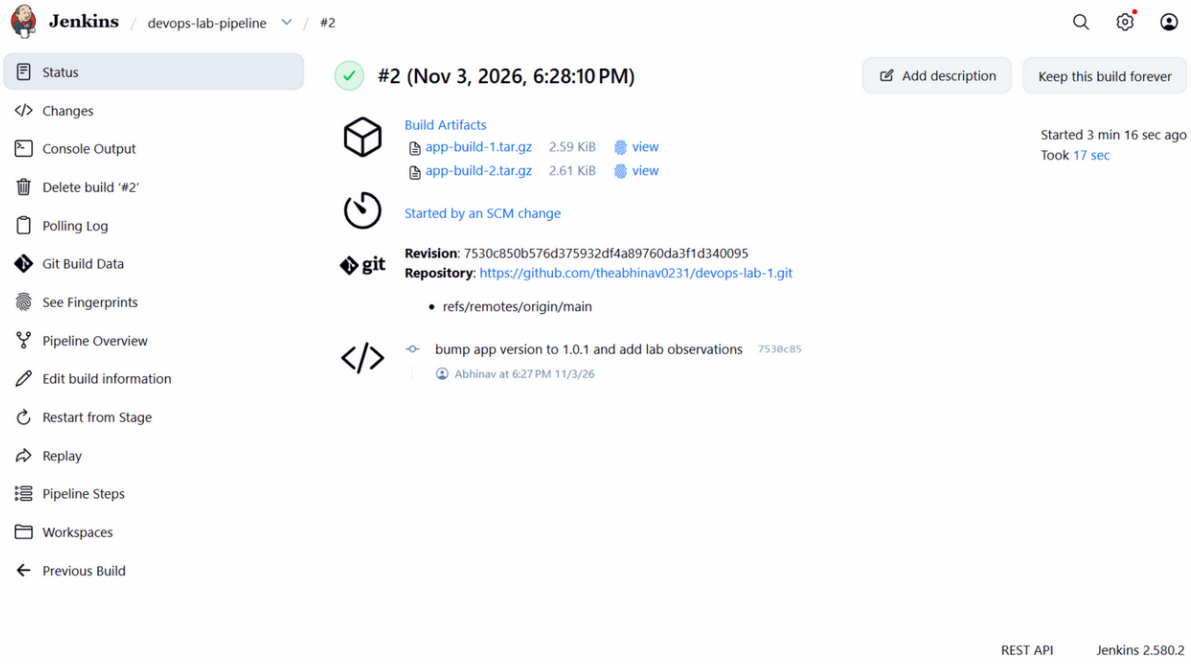


Fig: Build #2 (SCM triggered) with commit bump app version to 1.0.1

```
abhinav@LAPTOP-BSV7PCJK: docs/commands  —  terminal
Collecting iniconfig (from pytest==8.3.4->-r app/requirements.txt (line 2))
  Downloading iniconfig-2.3.1-py3-none-any.whl.metadata (2.8 kB)
Collecting packaging (from pytest==8.3.4->-r app/requirements.txt (line 2))
  Downloading packaging-26.3-py3-none-any.whl.metadata (3.5 kB)
Collecting pluggy<2,>=1.5 (from pytest==8.3.4->-r app/requirements.txt (line 2))
  Downloading pluggy-1.7.0-py3-none-any.whl.metadata (5.0 kB)
Collecting MarkupSafe>=2.0 (from Jinja2>=3.1.2->flask==3.1.0->-r app/requirements.txt (line 1))
  Downloading markupsafe-3.0.4-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_6
Downloading flask-3.1.0-py3-none-any.whl (102 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 103.0/103.0 kB 2.7 MB/s eta 0:00:00
Downloading pytest-8.3.4-py3-none-any.whl (343 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 343.1/343.1 kB 7.0 MB/s eta 0:00:00
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.5.0-py3-none-any.whl (125 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 125.3/125.3 kB 5.4 MB/s eta 0:00:00
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 134.9/134.9 kB 5.2 MB/s eta 0:00:00
Downloading pluggy-1.7.0-py3-none-any.whl (27 kB)
Downloading werkzeug-3.1.9-py3-none-any.whl (228 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 228.7/228.7 kB 7.6 MB/s eta 0:00:00
Downloading iniconfig-2.3.1-py3-none-any.whl (7.6 kB)
Downloading packaging-26.3-py3-none-any.whl (129 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 130.0/130.0 kB 5.4 MB/s eta 0:00:00
Downloading markupsafe-3.0.4-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.
Installing collected packages: pluggy, packaging, MarkupSafe, itsdangerous, iniconfig, click, blinker, Wer
Successfully installed Jinja2-3.1.6 MarkupSafe-3.0.4 Werkzeug-3.1.9 blinker-1.9.0 click-8.5.0 flask-3.1.0
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Running unit tests...
[Pipeline] sh
+ ./venv/bin/python -m pytest tests/ -v
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-8.3.4, pluggy-1.7.0 -- /var/lib/jenkins/workspace/devops-lab-pipel
cachedir: .pytest_cache
```

Fig: Console: package and deploy stages

```
abhinav@LAPTOP-BSV7PCJK: docs/commands  —  terminal
rootdir: /var/lib/jenkins/workspace/devops-lab-pipeline
collecting ... collected 2 items
tests/test_app.py::test_index_returns_message PASSED                    [ 50%]
tests/test_app.py::test_health_returns_up PASSED                        [100%]
============================== 2 passed in 0.15s ===============================
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Package)
[Pipeline] sh
+ mkdir -p dist
+ tar czf dist/app-build-1.tar.gz app tests
[Pipeline] archiveArtifacts
Archiving artifacts
Recording fingerprints
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploying app to /opt/devops-lab1 and restarting service...
[Pipeline] sh
+ sudo bash /var/lib/jenkins/workspace/devops-lab-pipeline/scripts/deploy.sh
Deploying from /var/lib/jenkins/workspace/devops-lab-pipeline to /opt/devops-lab1 ...
Deploy successful - health check returned HTTP 200
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline finished: BUILD SUCCESS
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
```

Fig: Console: post actions, Finished: SUCCESS

Observations

- Declarative syntax keeps the pipeline readable; each stage has a single responsibility.
- post { success / failure } prints a clear result line at the end of every run.
- A failing pytest or health check would stop the pipeline before the deploy stage or turn the build red, so broken code is never deployed.

Conclusion

All the commands were executed successfully on Ubuntu 24.04 LTS and the evidence (command logs, screenshots and commit history) is committed in the GitHub repository github.com/theabhinav0231/devops-lab-1 under the docs/ directory.